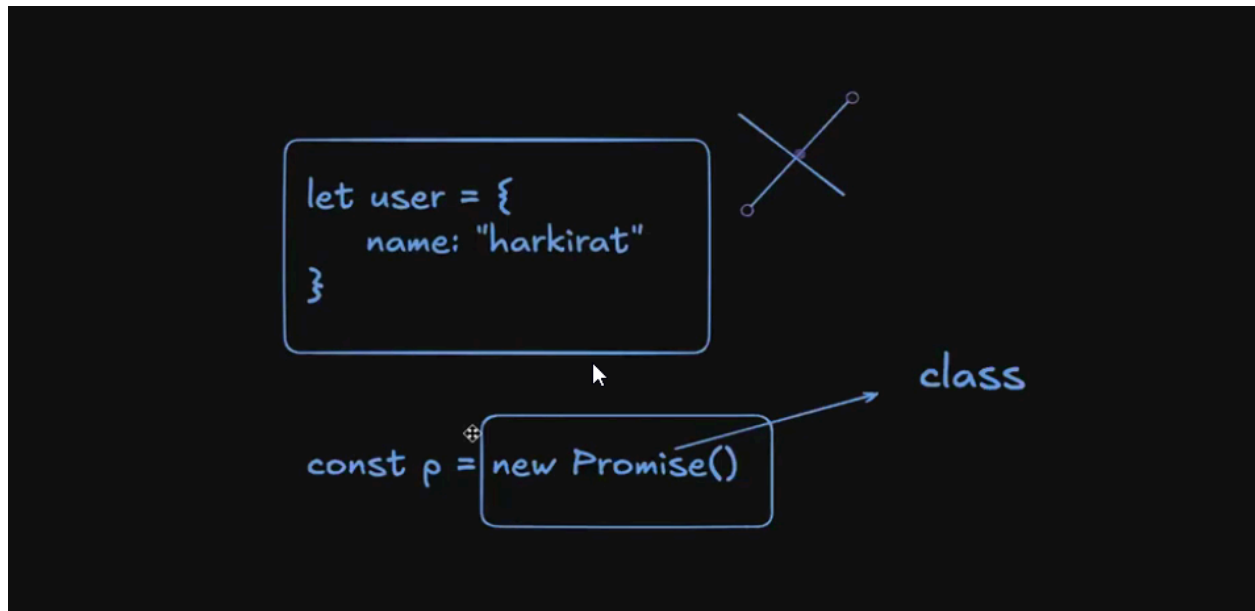




Asynchronous Operations

```
fs.readFile()  
setTimeout()  
fs.writeFile()  
-----  
database calls  
network calls
```

The diagram illustrates the difference between asynchronous and synchronous file operations. On the left, a box lists asynchronous methods: `fs.readFile()`, `setTimeout()`, and `fs.writeFile()`, followed by a dashed line and the text "database calls" and "network calls". A checkmark is placed next to this box. On the right, a code snippet shows `fs.readFile("a.txt", "utf-8", Function(err, data) {` with a blue highlight on the callback function. An arrow points from the top of the diagram to this code. Below the code, the text "119011" is visible. At the bottom, a code snippet shows `let data = fs.readFileSync("a.txt", "utf-8")` with a mouse cursor pointing at it.

```
fs.readFile()
setTimeout()
fs.writeFile()
-----
database calls
network calls
```

```
fs.readFile("a.txt", "utf-8", Function(err, data) {
```

```
let data = fs.readFileSync("a.txt", "utf-8")
```

callback | Promise | Async ... await

The diagram compares three asynchronous programming patterns. It is divided into three columns by vertical lines. The first column is titled "callback based async function" and lists `fs.readFile`, `fs.writeFile`, and `setTimeout`. The second column is titled "promises with the .then syntax". The third column is titled "async await syntax".

```
callback
based async function

fs.readFile
fs.writeFile
setTimeout
```

```
promises
with the
.then syntax
```

```
async await syntax
```

Approach #1 (callback based, sync fn calls)

```
1 const fs = require("fs");
2
3 function cleanFileSync(filePath, cb) {
4   const contents = fs.readFileSync(filePath, "utf-8");
5   const trimmedContents = contents.trim();
6   fs.writeFileSync("a.txt", trimmedContents)
7 }
8
9 cleanFileSync("a.txt");
```

Approach #2 (callback based, async function calls)

```
const fs = require("fs");

function cleanFile(filePath, cb) {
  fs.readFile(filePath, "utf-8", function (err, contents) {
    const trimmedContents = contents.trim();
    fs.writeFile("a.txt", trimmedContents, function() {
      cb();
    });
  });
}

cleanFile("a.txt", function() {
  console.log("done cleaning a.txt")
});
```

Approach #1 (callback based, sync fn calls)

```
1 const fs = require("fs");
2
3 function cleanFileSync(filePath, cb) {
4   const contents = fs.readFileSync(filePath, "utf-8");
5   const trimmedContents = contents.trim();
6   fs.writeFileSync("a.txt", trimmedContents)
7 }
8
9 cleanFileSync("a.txt");
```

Approach #2 (callback based, async function calls)

```
const fs = require("fs");

function cleanFile(filePath, cb) {
  fs.readFile(filePath, "utf-8", function (err, contents) {
    const trimmedContents = contents.trim();
    fs.writeFile("a.txt", trimmedContents, function() {
      cb();
    });
  });
}

cleanFile("a.txt", function() {
  console.log("done cleaning a.txt")
});
```

Approach #3 (promise based, no async await)

```
1 const fs = require("fs");
2
3 function cleanFile(filePath) {
4   return new Promise(function(resolve, reject) {
5     fs.readFile(filePath, "utf-8", function (err, contents) {
6       if (err) {
7         reject();
8       } else {
9         const trimmedContents = contents.trim();
10        fs.writeFile("a.txt", trimmedContents, function(err) {
11          if (err) {
12            reject();
13          } else {
14            resolve();
15          }
16        });
17      }
18    });
19  });
20 }
```

Approach #4 (promise based, does use async await)

```
1 const fs = require("fs");
2
3 function cleanFile(filePath) {
4   return new Promise(function(resolve, reject) {
5     fs.readFile(filePath, "utf-8", function (err, contents) {
6       if (err) {
7         reject();
8       } else {
9         const trimmedContents = contents.trim();
10        fs.writeFile("a.txt", trimmedContents, function(err) {
11          if (err) {
12            reject();
13          } else {
14            resolve();
15          }
16        });
17      }
18    });
19  });
20 }
21
22 async function main() {
23   try {
24     await cleanFile("a.txt");
25     console.log("done cleaning the file");
26   } catch (err) {
27     console.log(err);
28   }
29 }
```